Lektion 29

Normalisering

Mål

Efter endt undervisning kan den studerende:

- Normalisere tabeller op til boyce-codd
- Vide hvornår man skal normalisere og hvornår man ikke skal.

Agenda

1. Recap fra sidste gang
2. Første Normalform
3. Første Normalform
4. Første Normalform
5. Anden normaliseringsform
6. Kode eksempel
7. Tredje Normalform
8. Kode eksempel
9. Boyce-Cood
10. Kode eksempel
11. Opgave 1NF

Recap fra sidste gang

Projekt oplæg

Første Normalform

Krav

1. Ingen gentagene gruppering
2. atomar værdier
3. Tydelig primær nøgle®
4. Alle attributter skal afhænge af en PK

Tabel eksempel

BookId	Title	ISBN	Author	CopyNumber	Status	Condition	BookshelfsId
1	title1	231	me	1, 2, 3		m, nm, nm	1,1,2

Library
BookId
Title
ISBN
Author
{Copies}
CopyNumber
Status
Condition
BookshelfsId
Row
Size
LoanDate
ReturnDate
UserId
Name
Mail
Title

Første Normalform

Tabel eksempel

Rækkerne kan identificeres med en nøgle

<u>BookId</u>	Title	ISBN	Author	<u>CopyNumber</u>	Status	Condition	BookshelfsId
<u>1</u>	Title	123	me	<u>1</u>		m	1
<u>1</u>	Title	123	me	<u>2</u>		nm	1
<u>1</u>	Title	123	me	<u>3</u>		nm	2

Første Normalform

Kode eksempel

```
class LibraryCopyModel
{
    // Key
    public byte CopyNumber {get;set;}

    // Attributes
    public string Status {get;set;}
    public string Condition {get;set;}
    public byte BookshelfsId {get;set;}

    // Key
    public int BookId {get;set;}

    // Attributes
    public string Title {get;set;}
    public string ISBN {get;set;}
    public string Author {get;set;}
}
```

Anden normaliseringsform

Krav

1. 1 tabel -> 1 pk
2. Opklare partielle afhængigheder

Tabel eksempel

BookId	Title	ISBN	Author	CopyNumber	Status	Condition	BookshelfsId
1	Title	123	me	1		m	1
1	Title	123	me	2		nm	1
1	Title	123	me	3		nm	2

Library
BookId
Title
ISBN
Author
CopyNumber
Status
Condition
BookshelfsId
Row
Size
LoanDate
ReturnDate
UserId
Name
Mail
Title

Opklare partielle afhængigheder

BookId	Title	ISBN	Author
1	Title	123	me

BookId	CopyNumber	Status	Condition	BookshelfsId	Shelfs	Rooms
1	1		m	1	4	3
1	2		nm	1	4	3
1	3		nm	2	5	2

Kode eksempel

```
class BookCopyModel
{
    public byte CopyNumber {get;set;}
    public int BookId {get;set;}
    public string Status {get;set;}
    public string Condition {get;set;}
    public byte BookshelfsId {get;set;}
    public byte Shelfs {get;set;}
    public byte Rooms {get;set;}
}
class BookModel
{
  public int BookId {get;set;}
  public string Title {get;set;}
  public string ISBN {get;set;}
  public string Author {get;set;}
}
```

```
// BookCopy
modelBuilder.Entity<BooksCopyModel>()
    .HasKey(bc => new { bc.BookId, bc.CopyNumber });
// Book
modelBuilder.Entity<BooksModel>()
    .HasKey(b => b.BookId);
// BookCopy -> Book
modelBuilder.Entity<BooksCopyModel>()
    .HasOne(bc => bc.Book)
    .WithMany(b => b.BooksCopies)
    .HasForeignKey(bc => bc.BookId);
```

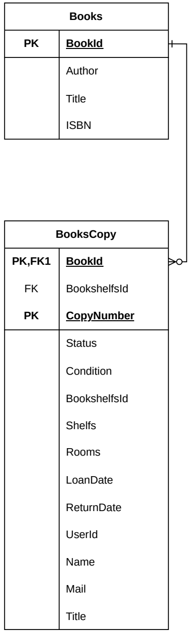
Tredje Normalform

Krav

1. Ny tabeller for transiente afhængigheder
2. Placer attributter de rigtig steder

BookId	Title	ISBN	Author
1	Title	123	me

BookId	CopyNumber	Status	Condition	BookshelfsId	Shelfs	Rooms
1	1		m	1	4	3
1	2		nm	1	4	3
1	3		nm	2	5	2



Afklare tilhørsforhold for attributter

BookId	Title	ISBN	Author
1	Title	123	me

BookId	CopyNumber	Status	Condition	BookshelfsId
1	1		m	1
1	2		nm	1
1	3		nm	2

BookshelfsId	Shelfs	Rooms
1	4	3

Kode eksempel

```
class BookCopyModel
{
    public byte CopyNumber {get;set;}
    public int BookId {get;set;}
    public string Status {get;set;}
    public string Condition {get;set;}
    public byte BookshelfsId {get;set;}
}
class BookModel
{
  public int BookId {get;set;}
  public string Title {get;set;}
  public string ISBN {get;set;}
  public string Author {get;set;}
}
class BookshelfsModel
{
    public byte BookshelfsId {get;set;}
    public byte Shelfs {get;set;}
    public byte Rooms {get;set;}
}
```

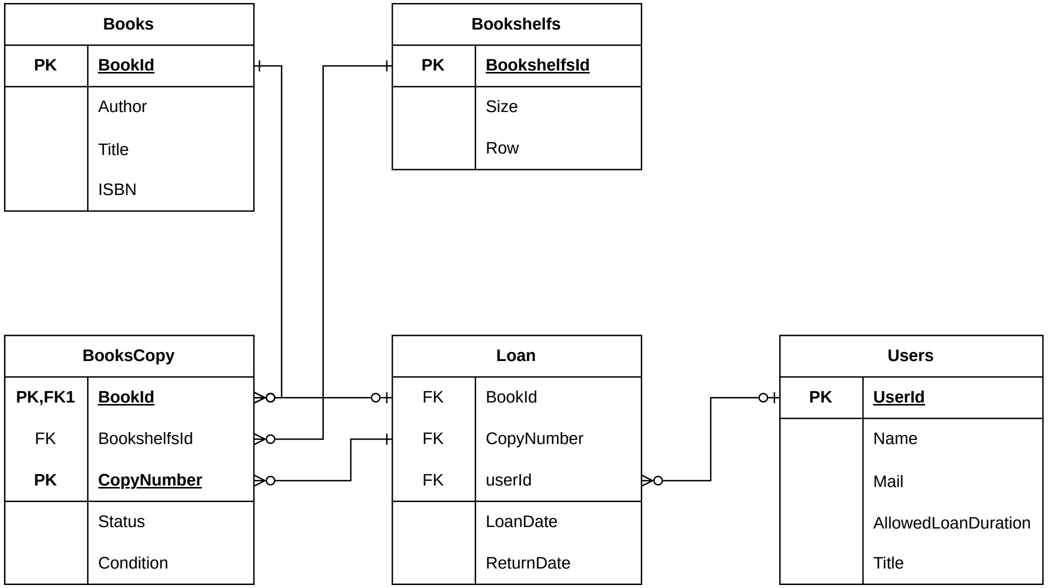
```
// BookCopy
modelBuilder.Entity<BooksCopyModel>()
    .HasKey(bc => new { bc.BookId, bc.CopyNumber });
// Book
modelBuilder.Entity<BooksModel>()
    .HasKey(b => b.BookId);
// BookCopy -> Book
modelBuilder.Entity<BooksCopyModel>()
    .HasOne(bc => bc.Book)
    .WithMany(b => b.BooksCopies)
    .HasForeignKey(bc => bc.BookId);
// Bookshelf
modelBuilder.Entity<BookshelfsModel>()
    .HasKey(bs => bs.BookshelfId);

// BookCopy -> Bookshelf
modelBuilder.Entity<BooksCopyModel>()
    .HasOne(bc => bc.Bookshelf)
    .WithMany(bs => bs.BooksCopies)
    .HasForeignKey(bc => bc.BookshelfId);
```

Boyce-Cood

Krav

1. Overholde 3nf for keys - altså ingen transiente candidate keys.



Afklare tilhørsforhold for attributter

UserId	TitleId	Name	Mail
1	1	Stefan	skqu@iba.dk
2	2	Not Stefan	nskqu@iba.dk

TitleId		
1	unlimited	Module Leader
2	2	Student

Kode eksempel

```
class UserModel
{
    public byte UserId {get;set;}
    public byte RoleId {get;set;}
    public string Name {get;set;}
    public string Mail {get;set;}
}
class RolesModel
{
    public byte RoleId {get;set;}
    public uint AllowedLoanDuration {get;set;}
    public string Title {get;set;}
}
```

```
// User
modelBuilder.Entity<UserModel>()
    .HasKey(u => u.UserId);

// Role
modelBuilder.Entity<RolesModel>()
    .HasKey(r => r.RoleId)
    .WithMany(bc => bc.Users)
    .HasForeignKey(u => u.UserId)
```

Opgave 1NF

Med udgangspunkt i datasæt Opfyld 1NF

Opgave 2NF

Med udgangspunkt i datasæt Opfyld 2NF

Opgave 3NF

Med udgangspunkt i datasæt Opfyld 3NF

Opgave BCNF

Med udgangspunkt i datasæt Opfyld BCNF

Opgave 5 - Integration

Med udgangspunkt i datasæt Opfyld 1NF